

Notes on the Architecture, Moves, Energies, and Metropolis Rule in **ParticlesMC**

1 Purpose of the Code

ParticlesMC is a Julia package for Monte Carlo simulations of particle and molecular systems. The package is built as a domain-specific layer on top of **Arianna**. In this architecture, **Arianna** provides the generic Monte Carlo simulation machinery, while **ParticlesMC** defines:

- what a particle or molecular system is;
- how pair and bonded energies are computed;
- which Monte Carlo moves are available;
- how proposed moves are applied and reverted;
- how configurations and trajectories are loaded and stored;
- how a user-facing TOML file is converted into a simulation.

2 High-Level Architecture

The main package entry point is:

```
src/ParticlesMC.jl
```

This file defines the module, imports **Arianna**, includes the other source files, exports the public API, and defines the command-line interface **particlesmc**.

2.1 Main Source Files

ParticlesMC.jl	Top-level module, exports, generic helpers, and the particlesmc command-line interface.
atoms.jl	Defines the Atoms system type for non-bonded particle systems and its energy calculation.
molecules.jl	Defines the Molecules system type for systems with molecular identity and bonds. It contains both bonded and non-bonded energy calculations.
orientation.jl	Defines replaceable molecular-orientation conventions, periodic molecular un-wrapping, and mass-weighted center-of-mass calculations.
models.jl	Defines interaction models and potentials, for example Lennard-Jones, smoothed Lennard-Jones, soft spheres, and generalized Kremer-Grest-type interactions.
neighbours.jl	Defines neighbor-list data structures used to reduce the cost of local energy calculations.

<code>moves.jl</code>	Defines Monte Carlo actions such as particle displacements, species swaps, and molecule flips. It also defines proposal policies and methods required by Arianna .
<code>utils.jl</code>	Defines the target Boltzmann weight, periodic boundary helpers, species-list helpers, and the energy callback.
<code>IO/IO.jl</code>	Defines loading and storing of configurations. Format-specific logic is split into <code>IO/xyz.jl</code> , <code>IO/exyz.jl</code> , and <code>IO/lammps.jl</code> .
<code>rotation.jl</code>	Defines a custom AriannaAlgorithm for molecular rotation observables.

2.2 Simulation Flow

The command-line path is:

1. read a TOML parameter file;
2. read initial configurations through `load_chains`;
3. build either **Atoms** or **Molecules** systems;
4. build the interaction model matrix;
5. construct a pool of moves;
6. construct an **Arianna** algorithm list;
7. create `Simulation(chains, algorithm_list, steps)`;
8. call `run!(simulation)`.

Conceptually:

TOML input $\rightarrow$ systems $\rightarrow$ moves + algorithms $\rightarrow$ Arianna simulation $\rightarrow$ outputs.

3 System Types

3.1 Particles

The abstract base type is:

```
abstract type Particles <: AriannaSystem end
```

Both **Atoms** and **Molecules** are subtypes of **Particles**. This lets generic methods dispatch on **Particles**, while specialized methods can dispatch on **Atoms** or **Molecules**.

3.2 Atoms

Atoms represents a non-bonded particle system. Important fields include:

- **position**: particle positions;
- **species**: particle species labels;
- **density, temperature**;
- **energy**: total system energy, stored as a one-element vector;
- **model_matrix**: pair interaction model for every species pair;
- **box**: periodic simulation box;
- **neighbour_list**: object used to find local neighbors;
- **species_list**: helper used for species-swap moves.

3.3 Molecules

`Molecules` extends the particle-system idea with molecular structure:

- `molecule`: molecule id for each particle;
- `mass`: mass of each particle;
- `start_mol`, `length_mol`: start and length of each molecule;
- `Nmol`: number of molecules;
- `bonds`: bonded neighbors for each particle;
- `Phi`: rotation-vector observable storage.

Mass is stored once per particle. The public `System` constructor accepts the optional keyword `masses`. If it is omitted, every particle receives unit mass, preserving the behavior of configurations created before mass support was introduced. Supplied masses must be finite, strictly positive, and have the same length as the position array. For command-line simulations, masses may be specified in the `[system]` TOML table:

```
[system]
masses = [1.0, 2.0, 1.0]
```

Mass does not modify the configurational pair Hamiltonian or the current Monte Carlo acceptance rule. It is presently used to define molecular centers of mass and orientations.

4 Molecular Orientation

Molecular orientation is implemented separately in `src/orientation.jl`. This separation allows the geometrical convention to change without modifying Monte Carlo moves or the future external-field energy term. All orientation functions return a unit vector and account for periodic boundaries by first placing the atoms in mutually consistent images.

For positions $\mathbf{r}_i$ and masses m_i , the center of mass is

$$\mathbf{r}_{\text{COM}} = \frac{\sum_i m_i \mathbf{r}_i}{\sum_i m_i}. \quad (1)$$

It is available through:

```
center = molecule_center_of_mass(system, molecule_id)
```

4.1 Center-to-Atom Convention

`CenterToAtomOrientation(a)` defines the orientation from the center of mass to local atom a :

$$\mathbf{n} = \frac{\mathbf{r}_a - \mathbf{r}_{\text{COM}}}{\|\mathbf{r}_a - \mathbf{r}_{\text{COM}}\|}. \quad (2)$$

For example:

```
definition = CenterToAtomOrientation(1)
n = orientation(definition, system, molecule_id)
```

The atom index is local to the selected molecule rather than a global particle index.

4.2 Triangular Plane-Normal Convention

`PlaneNormalOrientation(a,b,c)` defines the right-handed normal to the surface determined by three ordered local atoms:

$$\mathbf{n} = \frac{(\mathbf{r}_b - \mathbf{r}_a) \times (\mathbf{r}_c - \mathbf{r}_a)}{\|(\mathbf{r}_b - \mathbf{r}_a) \times (\mathbf{r}_c - \mathbf{r}_a)\|}. \quad (3)$$

The default is `PlaneNormalOrientation(1,2,3)`. Exchanging two indices reverses $\mathbf{n}$, so atom ordering fixes the sign needed by a polar field coupling. The plane normal itself is independent of the atomic masses. Collinear or coincident defining coordinates are rejected because they do not define a unique normal.

The external-field Hamiltonian and its contribution to trial-move energy changes are not yet implemented; the current code establishes only mass storage and the orientation geometry on which that later term will depend.

5 Interaction Models and Hamiltonians

The interaction models are defined in `src/models.jl`. The base types are:

```
abstract type Model end
abstract type DiscreteModel <: Model end
abstract type ContinuousModel <: Model end
```

For a pair of particles i, j , the code obtains the appropriate interaction by species:

$$M_{ij} = \text{model_matrix}_{s_i, s_j}.$$

Then it evaluates either:

$$U_{ij}^{\text{nb}} = V_{M_{ij}}(r_{ij}^2),$$

or, for bonded molecular pairs,

$$U_{ij}^{\text{b}} = V_{M_{ij}}^{\text{bond}}(r_{ij}^2).$$

Here r_{ij} is computed using the nearest-image convention for periodic boundary conditions.

5.1 Hamiltonian for Particle Systems

For an `Atoms` system, the Hamiltonian is purely pairwise and non-bonded:

$$H_{\text{atoms}}(\mathbf{r}, \mathbf{s}) = \sum_{1 \leq i < j \leq N} V_{s_i s_j}(r_{ij}^2) \mathbf{1}(r_{ij}^2 \leq r_{\text{cut}, s_i s_j}^2). \quad (4)$$

Equivalently, the code computes a local energy:

$$E_i = \sum_{j \in \mathcal{N}(i)} V_{s_i s_j}(r_{ij}^2), \quad (5)$$

where $\mathcal{N}(i)$ is supplied by the chosen neighbor list. The total energy is initialized as:

$$H_{\text{atoms}} = \frac{1}{2} \sum_{i=1}^N E_i. \quad (6)$$

The factor 1/2 appears because a pair i, j is counted once in E_i and once in E_j .

5.2 Hamiltonian for Molecular Systems

For a `Molecules` system, the Hamiltonian has bonded and non-bonded parts:

$$H_{\text{mol}} = H_{\text{bond}} + H_{\text{nonbond}}. \quad (7)$$

The bonded part is:

$$H_{\text{bond}} = \sum_{(i,j) \in \mathcal{B}} V_{s_i s_j}^{\text{bond}}(r_{ij}^2), \quad (8)$$

where $\mathcal{B}$ is the set of bonds.

The non-bonded part is:

$$H_{\text{nonbond}} = \sum_{\substack{1 \leq i < j \leq N \\ (i,j) \notin \mathcal{B}}} V_{s_i s_j}(r_{ij}^2) \mathbf{1}(r_{ij}^2 \leq r_{\text{cut}, s_i s_j}^2). \quad (9)$$

Locally, for one particle i , the code computes:

$$E_i = \sum_{j \in \mathcal{B}(i)} V_{s_i s_j}^{\text{bond}}(r_{ij}^2) + \sum_{j \in \mathcal{N}(i), j \notin \mathcal{B}(i)} V_{s_i s_j}(r_{ij}^2). \quad (10)$$

Then the initial total energy is again:

$$H_{\text{mol}} = \frac{1}{2} \sum_{i=1}^N E_i. \quad (11)$$

6 Where Energy is Computed in the Code

The generic entry point is in `src/ParticlesMC.jl`:

```
function compute_energy_particle(system::Particles, i::Int)
    return compute_energy_particle(system, i, system.neighbour_list)
end
```

6.1 Atoms Energy

For `Atoms`, the local particle energy is computed in `src/atoms.jl`:

```
function compute_energy_particle(system::Atoms, i, neighbour_list::NeighbourList)
    energy_i = zero(typeof(system.density))
    position_i = get_position(system, i)
    for j in neighbour_list(system, i)
        energy_i += compute_energy_ij(system, i, j, position_i)
    end
    return energy_i
end
```

The pair energy is:

```

function compute_energy_ij(system::Atoms, i, j, position_i)
    i == j && return zero(typeof(system.density))
    model_ij = get_model(system, i, j)
    position_j = get_position(system, j)
    r2 = nearest_image_distance_squared(position_i, position_j, get_box(system))
    r2 > cutoff2(model_ij) && return zero(typeof(system.density))
    return potential(r2, model_ij)
end

```

6.2 Molecules Energy

For Molecules, local energy is computed in `src/molecules.jl`:

```

function compute_energy_particle(system::Molecules, i, neighbour_list::NeighbourList)
    position_i = system.position[i]
    bonds_i = system.bonds[i]

    energy_i = compute_energy_bonded_i(system, i, position_i, bonds_i)
    for j in neighbour_list(system, i)
        energy_i += check_nonbonded_compute_energy_ij(system, i, j, position_i, bonds_i)
    end
    return energy_i
end

```

Bonded energy calls:

```
bond_potential(r2, model_ij)
```

Non-bonded energy calls:

```
potential(r2, model_ij)
```

7 Moves and the Arianna Interface

`src/moves.jl` defines the available Monte Carlo actions and how they interact with the **Arianna** engine.

7.1 Important Distinction

`moves.jl` does not implement the full Metropolis algorithm. Instead, it implements methods that **Arianna**'s Metropolis algorithm calls.

That division is:

Arianna	Metropolis loop, accept/reject logic, scheduling
ParticlesMC	moves, energy differences, proposal densities, reverts

7.2 The Central Hook

The main method connecting moves to **Arianna** is:

```

function Arianna.perform_action!(system::Particles, action::Action)
    e1, e2 = perform_action!(system, action)
    if isinf(e1) || isinf(e2)
        action.delta_e = zero(typeof(system.energy[1]))
    else
        action.delta_e = e2 - e1
        system.energy[1] += action.delta_e
    end
    return e1, e2
end

```

In the source code the field is written as `delta_e` using the Julia unicode symbol `delta_e`; in these notes we write it as ΔE .

The purpose is:

1. compute the energy before the proposed move, E_1 ;
2. apply the move;
3. compute the energy after the move, E_2 ;
4. store $\Delta E = E_2 - E_1$;
5. temporarily update the system energy.

If **Arianna** rejects the move, it calls the corresponding `Arianna.revert_action!` method.

7.3 Displacement Move

`Displacement` moves one particle:

$$\mathbf{r}'_i = \mathbf{r}_i + \boldsymbol{\delta}.$$

The proposal policy `SimpleGaussian` samples:

$$i \sim \text{Uniform}\{1, \dots, N\}, \quad \boldsymbol{\delta} \sim \mathcal{N}(0, \sigma^2 I).$$

Because the Gaussian displacement is symmetric,

$$q(x \rightarrow x') = q(x' \rightarrow x),$$

so the proposal-density terms cancel in ordinary Metropolis sampling.

The code computes only the local energy of particle i before and after the move:

$$\Delta E = E_i(\mathbf{r}'_i) - E_i(\mathbf{r}_i).$$

7.4 Discrete Species Swap

`DiscreteSwap` swaps the species labels of two particles:

$$s'_i = s_j, \quad s'_j = s_i.$$

The policy `DoubleUniform` selects one particle from each of two species lists:

$$i \sim \text{Uniform}(\{k : s_k = a\}), \quad j \sim \text{Uniform}(\{k : s_k = b\}).$$

The log proposal density in the code is:

$$\log q(x \rightarrow x') = -\log(N_a N_b),$$

where N_a and N_b are the numbers of particles of the two selected species.

The energy change is computed from the local energies of particles i and j :

$$\Delta E = (E'_i + E'_j) - (E_i + E_j).$$

7.5 Energy-Biased Swap

EnergyBias is a non-uniform proposal policy. It samples particles with weights depending on their local energies:

$$w_i^{(a)} \propto \exp(\theta_1 E_i), \quad w_j^{(b)} \propto \exp(\theta_2 E_j).$$

The proposal density is therefore not generally symmetric. This is why the code implements:

```
Arianna.log_proposal_density(action, ::EnergyBias, parameters, system)
```

For non-symmetric proposals, the Metropolis-Hastings correction must include the proposal-ratio term.

7.6 Molecule Flip

MoleculeFlip swaps two sites within the same molecule:

$$s'_i = s_j, \quad s'_j = s_i, \quad i, j \in \text{same molecule}.$$

It is similar to **DiscreteSwap**, but constrained by molecular membership.

8 Where the Metropolis Rule Is

The Metropolis algorithm itself is not defined in `src/moves.jl`. It is provided by **Arianna**. In this package, **Metropolis** appears when building the algorithm list:

```
(algorithm=Metropolis, pool=pool, seed=seed,
parallel=parallel, sweepstep=length(chains[1]))
```

The Boltzmann target density needed by Metropolis is defined in `src/utils.jl`:

```
function Arianna.unnormalised_log_target_density(e, system::Particles)
    return -e / system.temperature
end
```

and:

```
function Arianna.delta_log_target_density(e1, e2, system::Particles)
    return -(e2 - e1) ./ system.temperature
end
```

Thus, for a symmetric proposal, the acceptance probability is:

$$P_{\text{acc}}(x \rightarrow x') = \min \left[1, \exp \left(-\frac{\Delta E}{T} \right) \right]. \quad (12)$$

For a non-symmetric proposal, the Metropolis-Hastings form is:

$$P_{\text{acc}}(x \rightarrow x') = \min \left[1, \exp \left(-\frac{\Delta E}{T} + \log q(x' \rightarrow x) - \log q(x \rightarrow x') \right) \right]. \quad (13)$$

The proposal-density terms are supplied by `moves.jl` through methods like:

```
Arianna.log_proposal_density(action, policy, parameters, system)
```

9 Summary

- The full simulation loop and Metropolis accept/reject rule are provided by **Arianna**.
- **ParticlesMC** defines systems, energies, moves, proposal policies, and file IO.
- Energy is computed locally using neighbor lists.
- For **Atoms**, the Hamiltonian is a sum of non-bonded pair potentials.
- For **Molecules**, the Hamiltonian contains both bonded and non-bonded contributions.
- `moves.jl` tells **Arianna** how to sample, apply, score, and revert moves.
- The Boltzmann target used by Metropolis is $\log \pi(x) = -H(x)/T$.